

תכנות מקבילי

מערכות הפעלה

THREADS

THREADS

- Thread הוא ריצה של פקודות תוכנה
- כמו תהליך, אבל פשוט יותר
- אין לו שם מקובל בעברית אבל לפעמים קוראים לו חוט (thread בעברית) או תהליכון
- בעצם בהרצה סטנדרטית של תוכנה אנו מריצים thread יחיד
- ניתן גם לכתוב תוכנה שמריצה מספר threads שונים
- מאין "מיקרו-קוסמוס" של מערכת הפעלה – כמו שמערכת ההפעלה יכולה להריץ תהליכים שונים, תוכנה יכולה להריץ threads שונים
- בעוד שגם בתוכנות שראינו עד כה השתמשנו במחלקות ופונקציות כדי **לכתוב את הקוד** בצורה מודולרית, ההרצה של התוכנה עדיין רצה כרצף יחיד
- עבודה עם threads מאפשרת לנו לכתוב קוד **שירוצ** באופן מודולרי
- זה פותח בפנינו יכולות חדשות כגון תכנות מקבילי
- זה גם מציב בפנינו אתגרים שונים כגון ניהול משאבים תקין

THREADS

```
public class MyFirstThread extends Thread {  
    @Override  
    public void run() {  
        System.out.println("hello world");  
    }  
}
```

```
Thread t = new MyFirstThread();  
t.run();
```

hello world

- כדי להגדיר thread נגדיר טיפוס חדש אשר יורש מהמחלקה המובנית Thread
- בתוך המחלקה נדרוס את השיטה run()
- בשיטה זו נגדיר מה קורה כאשר ירוץ הthread
- בתוכנה הראשית נוכל לייצר אובייקט חדש מהטיפוס שהגדרנו
- וכדי להריץ נשתמש בשיטה run

פונקציה כערך

```
private static void printHelloWorld() {  
    System.out.println("hello world");  
}  
private static void rollADice() {  
    System.out.println(1+(int)(Math.random()*6));  
}
```

```
String[] descriptions = {  
    "print \"hello world\"",  
    "roll a dice"  
};  
System.out.println("choose option:");  
for (int i = 0; i < descriptions.length; i++ )  
    System.out.println(i+":\t"+descriptions[i]);  
int opt = in.nextInt();  
if (opt == 0) printHelloWorld();  
else if (opt == 1) rollADice();
```

- כיוון שthread הוא אובייקט שמריץ פונקציה, זה בעצם מאפשר לנו ב"אינטואיטיבית" להעביר פונקציה כערך
- לדוגמא נניח שאנו רוצים לכתוב תוכנה שפותחת למשתמש תפריט שבו יבחר בין שתי פונקציות: הדפסה של hello world, והטלת קוביה (הדפסת מספר רנדומלי בין 1 ל 6)
- ללא threads:
- כעת אם נרצה להוסיף עוד אופציות נצטרך לערוך את כל הקוד

פונקציה כערך

```
public class DiceRoller extends Thread {  
    @Override public void run()  
    { System.out.println(1+(int)(Math.random()*6)); }  
}
```

```
public class Printer extends Thread {  
    @Override public void run()  
    { System.out.println("hello world"); }  
}
```

```
String[] descriptions = {  
    "print \"hello world\"",  
    "roll a dice",  
};  
Thread[] functions = {  
    new Printer(),  
    new DiceRoller(),  
};  
System.out.println("choose option:");  
for (int i = 0; i < descriptions.length; i++)  
    System.out.println(i+":\t"+descriptions[i]);  
functions[in.nextInt()].run();
```

- כיוון שthread הוא אובייקט שמריץ פונקציה, זה בעצם מאפשר לנו ב"אינטואיטיבית" להעביר פונקציה כערך

- לדוגמא נניח שאנו רוצים לכתוב תוכנה שפותחת למשתמש תפריט שבו יבחר בין שתי פונקציות: הדפסה של hello world, והטלת קוביה (הדפסת מספר רנדומלי בין 1 ל 6)

- עם threads:

- כעת אם נרצה להוסיף עוד אופציות נצטרך רק להוסיף Thread חדש למערך

פונקציה כערך

- ניתן גם להשתמש בזה כדי "לייצר" פונקציה (כלומר thread שיריץ אותה)

- לדוגמא: פונקציה המקבלת ערך מינימום וערך מקסימום ומחזירה "פונקציה" שמדפיסה הגרלה של

המספר בטווח

```
public class RandomizerThread extends Thread {  
    private int min,max;  
    public RandomizerThread(int min, int max) {  
        this.min = min;  
        this.max = max;  
    }  
    @Override public void run() {  
        System.out.println(min+(int)(Math.random()*(max-min+1)));  
    }  
    private static Thread generateRandoizer(int min, int max) {  
        return new RandomizerThread(min,max);  
    }  
}
```

```
Thread t = generateRandoizer(5,10);  
t.run();t.run();  
t = generateRandoizer(20,25);  
t.run();t.run();
```

8
5
20
21

פונקציה כערך

- פונקציה המשמשת בפונקציה אחרת כערך (מקבלת, מחזירה, ...) נקראת פונקציה מסדר גבוהה (High-Order Function)
- פונקציות מסדר גבוה מהוות רכיב עיקרי בשפות תכנות פונקציונליות – משפחה של שפות תכנות בעלות מבנה שונה לשפות תכנות אימפרטיביות (שפות תכנות "רגילות") – אשר כוללת פונקציות כערכים בכחלק בסיסי מתחביר השפה
- גם שפות אימפרטיביות רבות (JAVA לא בינהן) מספקות תמיכה בפונקציות מסדר גבוה בצורה מובנית
- Threads מספקים לנו דרך להגדיר פונקציות מסדר גבוה, למרות שבצורה מאוד מוגבלת – ניתן רק פונקציות שלא מקבלות כלום ולא מחזירות כלום, והמנגון לייצר אותם דרך threads מסורבל
- אמנם JAVA לא מספקת תמיכה מובנית בפונקציות כערכים אבל יש חבילות שמוסיפות תמיכה בכך בצורה הרבה יותר עשירה ונוחה לשימוש

תכנות מקבילי

תכנות מקבילי

```
Thread t1 = new MyFirstThread();  
Thread t2 = new MyFirstThread();  
t1.start();t2.start();
```

```
hello world  
hello world
```

- כפי שראינו ניתן להשתמש בשיטה run כדי להריץ thread
- ניתן גם להשתמש בשיטה start כדי להתחיל את ההרצה בלי לחכות לסופה
- באופן זה אנו יכולים להריץ מספר threads במקביל
- כיוון שכל thread מריץ פקודה אחת, אנו לא ממש רואים את ההרצה המקבילית
- נעדכן את הthread כך שידפיס כל תו בנפרד

```
@Override public void run() {  
    char[] cs = "hello world how are you?".toCharArray();  
    for (char c : cs)  
        System.out.print(c);  
    System.out.println();  
}
```

```
hello world how are youhello world how?  
→w are you?
```

תכנות מקבילי

```
@Override synchronized public void run() {  
    char[] cs = "hello world how are you?"  
        .toCharArray();  
    for (char c : cs) {  
  
        System.out.print(c);  
        try {  
            wait(1);  
        } catch (InterruptedException e) { }  
    }  
    System.out.println();  
}
```

hheellllloo wwoorrllldd hhooww aarree yyooouu??

- יש גם wait() שלא מקבל זמן – זה יגרום לתהליך לחכות עד שמשהו יגרום לו להמשיך (נראה בהמשך)

- thread ניתן לשלוט בצורה יותר מדוייקת בריצה
- כדי לעשות זאת צריך להגדיר את פונקצית הrun כsynchronized (בהמשך נראה שימושים נוספים לsynchronized)
- השיטה wait(int) מקבלת מספר מילי-שניות ומחכה את הזמן הזה (בזמן הזה, threads אחרים יכולים לרוץ)
- נעדכן את הthread כך שיחכה מילי-שנייה בין כל הדפסה של תו
- שימו לב שיש לעטוף את wait בtry/catch
- ניתן גם להשתמש בשיטה הסטטית Thread.sleep(int) (לא מצריך synchronized)

תכנות מקבילי

```
public class CharPrinter extends Thread {
    private char c;
    private int amount, waitTime;
    public CharPrinter(char c, int amount, int waitTime) {
        this.c = c;
        this.amount = amount;
        this.waitTime = waitTime;
    }
    @Override
    synchronized public void run() {
        for (int i = 0; i < amount; i++) {
            System.out.print(c);
            try {
                wait(waitTime);
            } catch (InterruptedException e) { }
        }
    }
}
```

```
Thread t1 = new CharPrinter('#', 10, 1);
Thread t2 = new CharPrinter('!', 7, 3);
t1.start(); t2.start();
```

- דוגמא: תוכנה בה רצים שני threads במקביל
- אחד מדפיס את התו # 10 פעמים, כאשר הוא מחכה מילי-שניה בין כל הדפסה
- השני מדפיס את התו ! 7 פעמים, כאשר הוא מחכה 3 מילי-שניות בין כל הדפסה

!###!###!##!##!!!

תכנות מקבילי

- נחזור לגרסא הפשוטה של thread ולהרצה אחת שלו
- ננסה להדפיס הודעה כשהוא מסיים
- מה נקבל?
- ההדפסה התבצעה לפני שהוא סיים!
- למה?
- התוכנה התחילה הרצה של thread והמשיכה
- פקודה ההדפסה הופעלה
- לאובייקט thread יש את השיטה isAlive שבודקת האם הוא עדיין בריצה
- נוכל להשתמש בה כדי לחכות עד שהוא ייסתיים

```
public class MyFirstThread extends Thread {  
    @Override public void run() {  
        System.out.println("hello world");  
    }  
}
```

```
Thread t = new MyFirstThread();  
t.start();  
System.out.println("done");
```

```
done  
hello world
```

```
Thread t = new MyFirstThread();  
t.start();  
while(t.isAlive());  
System.out.println("done");
```

```
hello world  
done
```

תכנות מקבילי

- לחכות בעזרת לולאה זו שיטה יקרה - המחשב כל הזמן מריץ בדיקות
- במקום זה אפשר להשתמש בשיטה join

```
public static void main(String[] args) throws InterruptedException {  
    Thread t = new MyFirstThread();  
    t.start();  
    t.join();  
    System.out.println("done");  
}
```

```
hello world  
done
```

תכנות מקבילי

- אנו רוצים להריץ שני threads:

1. ימלא מערך בהטלות קוביה (ערך רנדומלי בין 1 ל 6)

2. ידפיס את ערכי המערך וסכומו לאחר שהוא התמלא

- ניסיון ראשון

תכנות מקבילי

• Thread הממלא את המערך

```
public class ArrayFiller extends Thread {  
    private int[] arr;  
    public ArrayFiller(int[] arr) {  
        this.arr = arr;  
    }  
    @Override  
    public void run() {  
        for (int i = 0; i < arr.length; i++) {  
  
            arr[i] = 1+(int)(Math.random()*6);  
            System.out.println("put "+arr[i]+" at arr["+i+"]");  
            try { Thread.sleep(5); }  
            catch (InterruptedException e) { }  
        }  
    }  
}
```

לגרום לכך שהתליך ייקח זמן

תכנות מקבילי

• Thread המדפיס וסוכם

```
public class PrintAndSum extends Thread {  
    private int[] arr;  
    public PrintAndSum(int[] arr) {  
        this.arr = arr;  
    }  
    @Override  
    public void run() {  
        int sum = 0;  
        System.out.print("[ ");  
        for (int x : arr) {  
            sum+=x;  
            System.out.print(x+" ");  
        }  
        System.out.println("] sum="+sum);  
    }  
}
```

תכנות מקבילי

```
int[] arr = new int[3];  
Thread fill = new ArrayFiller(arr);  
Thread print = new PrintAndSum(arr);  
fill.start();  
print.start();  
while(print.isAlive());  
System.out.println("done");
```

```
[ 3 0 0 put 3 at arr[0]  
] sum=3  
done  
put 4 at arr[1]  
put 6 at arr[2]
```

- הרצה
- מה נקבל?
- מה קרה?
- thread הסוכם ביצע את החישובים לפני שהממלא סיים
- אנחנו פוגשים פה את אחד האתגרים של תכנות מקבילי – שיתוף משאבים

תכנות מקבילי

- כאשר שני threads רצים במקביל, פעמים רבות יהיה להם משאב משותף (shared resource)
- משאב משותף הוא משאב (כגון מבנה נתונים) אשר שני threads ניגשים אליו
- בעבודה עם משאב משותף, חשוב להגדיר את הthreads בזהירות, כדי לוודא התנהגות תקינה
- נכיר מספר דרכים לעשות זאת, ביניהם:
- סנכרון
- משאבים אטומיים

סנכרון

סנכרון

```
synchronized (SHARED_RESOURCE) {  
    //Code  
}
```

- בתוך קוד ניתן לפתוח בלוק סנכרון
- הקוד שבתוך הבלוק יהיה מסונכן על המשאב המשותף
- משמעות הדבר היא ששני threads לא יכולים להיות בו זמנית בזמן שהם פעילים בתוך הבלוק
- כלומר: כאשר thread נכנס לבלוק סנכרון על משאב
- אם הוא יכול לתפוס אותו, הוא תופס אותו וממשיך לרוץ
- אם הוא לא יכול (בלוק אחר כבר מחזיק בו), הוא מחכה עד שהמשאב יתפנה – או ששמי שמחזיק בו יצא מהבלוק או שיהפוך ללא פעיל (למשל – יכנס למצב המתנה)
- כאשר thread מחזיק במשאב הוא יכול להפעיל מתוכו את השיטות הבאות:
- wait() – thread בו הפעלנו יחכה עד ש"יקרה משהו" עם המשאב
- notify() – מודיע **לאחד** threads שמחכים למשאב ש"קרה משהו" והוא יכול להמשיך
- notifyAll() – מודיע **לכל** thread שמחכה למשאב ש"קרה משהו" והוא יכול להמשיך

סנכרון

- נחזור לדוגמא שלנו - אנו רוצים להריץ שני threads:
- 1. ימלא מערך בהטלות קוביה (ערך רנדומלי בין 1 ל 6)
- 2. ידפיס את ערכי המערך וסכומו לאחר שהוא התמלא
- ניסיון שני:
- נסנכרן את ה threads על המערך
- ה thread המדפיס יחכה עד שיקרה משהו במערך
- ה thread הממלא יודיע כאשר הוא יסיים למלא אותו

סכרון

- Thread הממלא את המערך

```
//ArrayFiller
@Override
public void run() {
    synchronized (arr) {
        for (int i = 0; i < arr.length; i++) {
            arr[i] = 1+(int)(Math.random()*6);
            System.out.println("put "+arr[i]+" at arr["+i+"]");
            try { Thread.sleep(5); }
            catch (InterruptedException e) { System.out.println(e);}
        }
        arr.notify();
    }
}
```

סכרון

• Thread המדפיס וסוכם

```
//PrintAndSum
@Override
public void run() {
    synchronized (arr) {
        try { arr.wait(); }
        catch (InterruptedException e) { System.out.println(e);}
        int sum = 0;
        System.out.print("[ ");
        for (int x : arr) {
            sum+=x;
            System.out.print(x+" ");
        }
        System.out.println("] sum="+sum);
    }
}
```


סנכרון

```
int[] arr = new int[3];
Thread fill = new ArrayFiller(arr);
Thread print = new PrintAndSum(arr);
fill.start();
print.start();
while(print.isAlive());
System.out.println("done");
```

```
put 6 at arr[0]
put 2 at arr[1]
put 4 at arr[2]
[ 6 2 4 ] sum=12
```

```
done
put 2 at arr[0]
put 5 at arr[1]
put 6 at arr[2]
```

• הרצה

• מה נקבל?

• לפעמים

• אבל לפעמים

• מה קורה?

• לפעמים הכל רץ תקין, אבל לפעמים thread הסוכם נשאר בהמתנה "לנצח"

• בעיה קלאסית בסנכרון, סדר האירועים משפיע על התוצאה:

• לפעמים thread הסוכם הפעיל את השיטה wait לפני שהthread הממלא הודיע על סיום – אז הכל תקין

• אבל לפעמים thread הממלא מפעיל notify לפני שהthread הסוכם אפילו להספיק להפעיל את wait, ואז הוא לא מקבל את ההודעה שתעיר אותו

• מצב זה הוא דוגמא לdeadlock: מצב שבו יש thread (או כמה threads) "תקוע" – הוא בהמתנה אבל אין שום דבר שיכול להעיר אותו

סכרון

- נחזור לדוגמא שלנו - אנו רוצים להריץ שני threads:

1. ימלא מערך בהטלות קוביה (ערך רנדומלי בין 1 ל 6)

2. ידפיס את ערכי המערך וסכומו לאחר שהוא התמלא

- ניסיון שלישי:

- כדי למנוע את הdeadlock נרצה שהthread המדפיס לא יבצע wait לאחר שהthread הממלא סיים

- נוסיף לthread הממלא דרך לבדוק אם הוא סיים

- בthread המדפיס ניכנס לwait רק אם הוא סיים

סכרון

```
//ArrayFiller
private boolean done = false;
public boolean isDone() {
    return done;
}
@Override
public void run() {
    synchronized (arr) {
        for (int i = 0; i < arr.length; i++) {
            arr[i] = 1+(int)(Math.random()*6);
            System.out.println("put "+arr[i]+" at arr["+i+"]");
            try { Thread.sleep(5); }
            catch (InterruptedException e) { System.out.println(e);}
        }
        done = true;
        arr.notify();
    }
}
```

Thread •
הממלא את
המערך

סכרן

Thread •
המדפיס
וסוכם

```
//PrintAndSum
private ArrayFiller filler;
public PrintAndSum(int[] arr,ArrayFiller filler) {
    this.arr = arr;
    this.filler = filler;
}
@Override
public void run() {
    synchronized (arr) {
        while (!filler.isDone()) {
            try { arr.wait(); }
            catch (InterruptedException e) { System.out.println(e);}
        }
        int sum = 0;
        System.out.print("[ ");
        for (int x : arr) {
            sum+=x;
            System.out.print(x+" ");
        }
        System.out.println("] sum="+sum);
    }
}
```

סנכרון

- שימו לב שהשתמשנו בwhile ולא בif למקרה שבו הthread יתעורר לפני שfiller סיים

```
while (!filler.isDone()) {  
    try { arr.wait(); }  
    catch (InterruptedException e) { System.out.println(e);}  
}
```

- בעוד שעקרונית, לא נראה שזה יכול לקרות אצלנו זה חשוב כי:
- בעתיד אולי נרצה להרחיב את המערכת למצב בו זה כן יתאפשר
- גם במצב הנוכחי, יכול להיות שמשהו שלא צפינו יגרום לו להתעורר

סכרון

• הרצה

```
int[] arr = new int[3];  
ArrayFiller fill = new ArrayFiller(arr);  
Thread print = new PrintAndSum(arr,fill);  
fill.start();  
print.start();  
while(print.isAlive());  
System.out.println("done");
```

```
put 5 at arr[0]  
put 2 at arr[1]  
put 2 at arr[2]  
[ 5 2 2 ] sum=9  
done
```

סכרון

- שימו לב להבדל בין notify ו notifyAll
- NotifyAll מודיע **לכל** threads שמחכים
- Notify מודיע **לאחד** מה threads שמחכים
- משמעות הדבר היא שאם מספר threads מחכים למשאב מסויים אבל התבצע עליו notify אחד, הם עשויים להישאר לא פעילים בלי שמשהו יעיר אותם
- שימו לב: ב JAVA יש מנגון שמאפשר לו לבצע הערה של threads גם ללא notify, לכן במקרים מסויימים בעיה זו לא תתרחש
- זו אחת הסיבות שבדוגמא הקודמת השתמשנו ב while ולא ב if
- כדי לבחון את ההבדל נגדיר שני סוגי threads
- Consomuer אשר ינעל על משאב ויחכה שהוא ישתחרר – נריץ שניים כאלו
- Filler אשר יסמן שהמשימה בוצעה וישחרר את המשאב - ננסה פעם אחת עם notify, ופעם אחת עם notifyAll- ונבחן את ההבדל

סכרון

- מחלקת עזר – DoneFlag – threads ישתמשו בה כדי לסמן מתי filler סיים

```
public class DoneFlag {  
    private boolean done = false;  
    public boolean isDone() {  
        return done;  
    }  
    public void setDone(boolean done) {  
        this.done = done;  
    }  
}
```


סכרון

- המחלקה Consumer – thread שמחכה שfiller יסיים. יכול
- שם
- מנעול – שדה מטיפוס Object עליו יתבצעו הנעילות
- את האובייקט DoneFlag
- בהרצה – יחכה על המנעול כל עוד DoneFlag לא סיים

```
public class Consumer extends Thread {  
    private final Object lock;  
    private final String name;  
    private final DoneFlag flag;  
    public Consumer(String name, Object lock, DoneFlag flag) {  
        this.name = name;  
        this.lock = lock;  
        this.flag = flag;  
    }  
    @Override  
    public void run() {  
        synchronized (lock) {  
            while (!flag.isDone()) {  
                try {  
                    System.out.println(name + " waiting...");  
                    lock.wait();  
                } catch (InterruptedException e) { }  
            }  
            System.out.println(name + " woke up and is working");  
        }  
    }  
}
```

סכרון

- המחלקה Filler – thread ש"עובד". יכול
- מנעול – שדה מטיפוס Object עליו יתבצעו הנעילות
- את האובייקט DoneFlag
- בהרצה – יסמן שהסתיים, ויעיר את מי שמחכה על המנעול (בתור התחלה עם notify)

```
public class Filler extends Thread {  
    private final Object lock;  
    private final DoneFlag flag;  
    public Filler(Object lock, DoneFlag flag) {  
        this.lock = lock;  
        this.flag = flag;  
    }  
    @Override  
    public void run() {  
        synchronized (lock) {  
            System.out.println("Filler doing work...");  
            flag.setDone(true);  
            System.out.println("Filler finished, notifying...");  
  
            lock.notify();  
            // lock.notifyAll();  
        }  
    }  
}
```

סנכרון

- הרצה – נריץ שני Consumer, ניתן להם זמן להתחיל ונריץ Filler

```
Object lock = new Object();
DoneFlag flag = new DoneFlag();
Consumer c1 = new Consumer("C1", lock, flag);
Consumer c2 = new Consumer("C2", lock, flag);
c1.start();c2.start();

// Give consumers time to start and wait
Thread.sleep(100);

Filler filler = new Filler(lock, flag);
filler.start();
```

```
C1 waiting...
C2 waiting...
Filler doing work...
Filler finished, notifying...
C1 woke up and is working
```

- C2 לא מתעורר

סנכרון

```
//lock.notify();  
lock.notifyAll();
```

```
C1 waiting...  
C2 waiting...  
Filler doing work...  
Filler finished, notifying...  
C1 woke up and is working  
C2 woke up and is working
```

- נשנה את filler שיפעיל notifyAll
- בהרצה:
- שניהם סיימו

משאבים אטומיים

משאבים אטומיים

- נגדיר מספר threads שעובדים על משאב משותף – מונה
- מחלקה עבור המונה

```
public class Counter {  
    private int val = 0;  
    public int plus1() {  
        int newvalue = val+1;  
        try { Thread.sleep(1); }  
        catch (InterruptedException e) { }  
        val = newvalue;  
        return newvalue;  
    }  
}
```

בלי זה זה ירוץ מהר
מדי כדי לבחון את
התופעה הלוונטית

משאבים אטומיים

- מחלקה עבור הthread שמגדיל

```
public class Increaser extends Thread {  
    public final String name;  
    private Counter counter;  
    public Increaser(String name, Counter counter) {  
        this.name = name;  
        this.counter = counter;  
    }  
    @Override  
    public void run() {  
        for (int i = 0; i < 3; i++)  
            System.out.println(name + " increased to " + counter.plus1());  
        super.run();  
    }  
}
```

משאבים אטומיים

```
Counter c = new Counter();  
for (int i = 0; i < 3; i++)  
    (new Increaser("I"+i, c)).start();
```

```
I2 increased to 1  
I1 increased to 1  
I0 increased to 1  
I1 increased to 2  
I0 increased to 2  
I2 increased to 2  
I2 increased to 3  
I0 increased to 3  
I1 increased to 3
```

```
public int plus1() {  
    int newvalue = val+1;  
    try { Thread.sleep(1); }  
    catch (InterruptedException e) { }  
    val = newvalue;  
    return newvalue;  
}
```

- הרצה
- מה קרה?
- כל thread ביצע הגדלה, אבל הן התבצעו במקביל
- במקום שאחד יגדיל מ 0 ל 1 ואז אחר מ 1 ל 2 וכך הלאה:
- כל אחד משלושת ה threads קרא שיש 0, בחר להגדיל ל 1, ואז כולם עדכנו ל 1
- כדי להמינע מכך אנו צריכים משאב אטומי

משאבים אטומיים

- פעולה נקראת אטומית אם היא מתבצעת "ברצף" בלי ש threads אחרים אולי יתבצעו במהלכה
- משאב נקרא אטומי עם כל פעולה שאפשר לבצע איתו היא אטומית
- נרצה לעדכן את counter כך שיהיה אטומי
- כפי שראינו ניתן להגדיר ששיטה היא synchronized
- זה גורם לכך שכאשר שיטה מופעלת, האובייקט ממנו נקרא השיטה ננעל, ואף thread אחר לא יוכל לגשת אליו (כמו לשים את כל קוד הפונקציה בתוך בלוק סנכרון על this)

משאבים אטומיים

```
public class Counter {  
    private int val = 0;  
    synchronized public int plus1() {  
        int newValue = val+1;  
        try { Thread.sleep(1); }  
        catch (InterruptedException e) { }  
        val = newValue;  
        return newValue;  
    }  
}
```

- נעדכן את Counter

- כעת בהרצה

```
I1 increased to 3  
I2 increased to 2  
I0 increased to 1  
I1 increased to 4  
I0 increased to 5  
I2 increased to 6  
I0 increased to 7  
I1 increased to 8  
I2 increased to 9
```

משאבים אטומיים

- JAVA מספקת מספר טיפוסים המיועדים לייצרת משאבים אטומיים
- יש לייבא אותם מהחבילה `java.util.concurrent.atomic`
- במידה ואחד מהם מתאים לצרכים שלנו נוכל להתשמש בו כדי לייצג משאב אטומי
- בדוגמא שלנו נוכל להשתמש ב- `AtomicInteger` – אשר מייצג מספר שלם אטומי
- נשתמש בו במקום המחלקה `Counter`

משאבים אטומיים

Increaser •

```
import java.util.concurrent.atomic.AtomicInteger;
public class Increaser extends Thread {
    public final String name;
    private AtomicInteger counter;
    public Increaser(String name, AtomicInteger counter) {
        this.name = name;
        this.counter = counter;
    }
    @Override
    public void run() {
        for (int i =0; i < 3; i++)
            System.out.println(name + " increased to "+counter.incrementAndGet());
        super.run();
    }
}
```

משאבים אטומיים

• הרצה

```
AtomicInteger c = new AtomicInteger();  
for (int i = 0; i < 3; i++)  
    (new Increaser("I"+i, c)).start();
```

```
I0 increased to 1  
I0 increased to 4  
I0 increased to 5  
I2 increased to 2  
I1 increased to 3  
I2 increased to 6  
I2 increased to 8  
I1 increased to 7  
I1 increased to 9
```